



CPU SIM TASK

Prepared by :

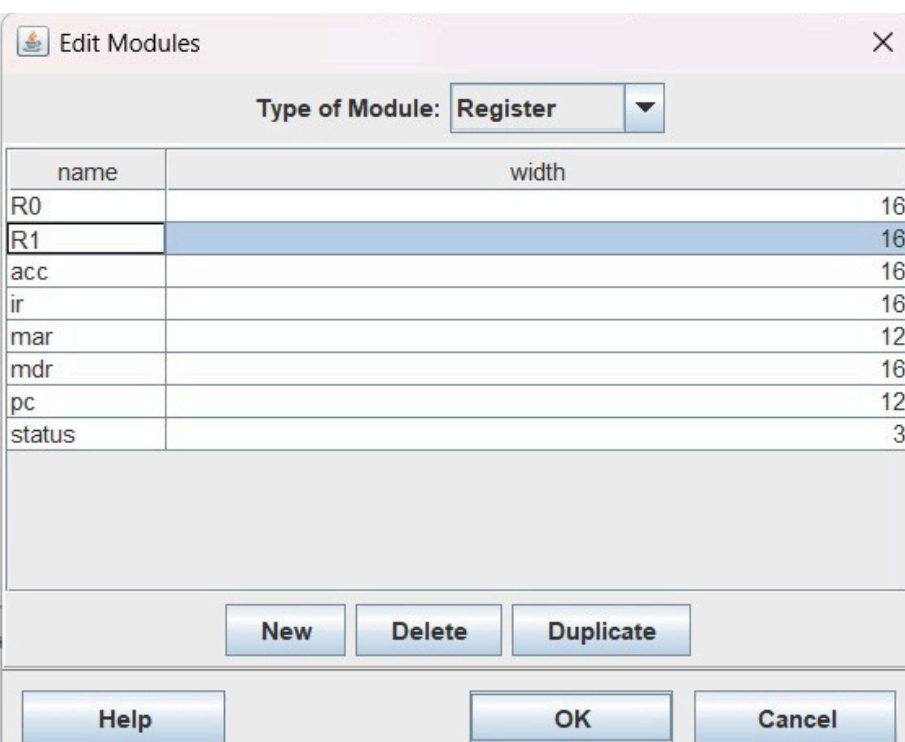
Team 10

- **Ahmed Elsherbiny**
- **Essam Mohammed**
- **Mariam Osman**
- **Merna Atef**



Hardware Part:

Hardware Modules



Edit Modules

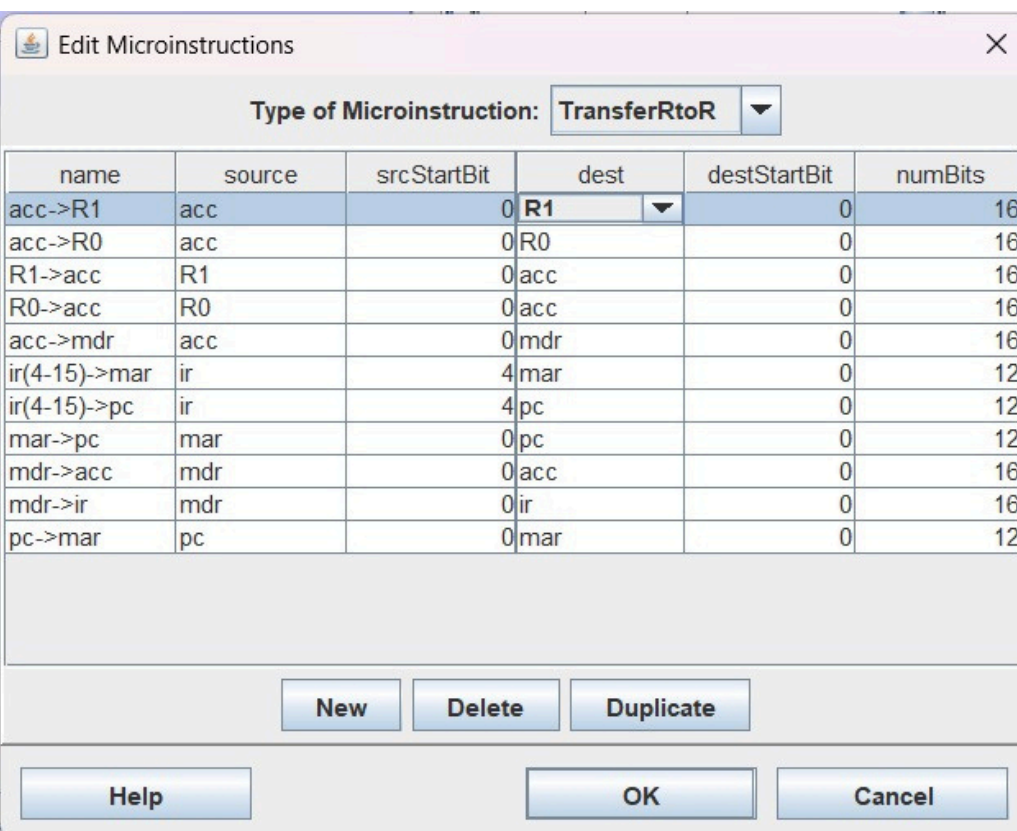
Type of Module: **Register**

name	width
R0	16
R1	16
acc	16
ir	16
mar	12
mdr	16
pc	12
status	3

New **Delete** **Duplicate**

Help **OK** **Cancel**

Microinstructions



Edit Microinstructions

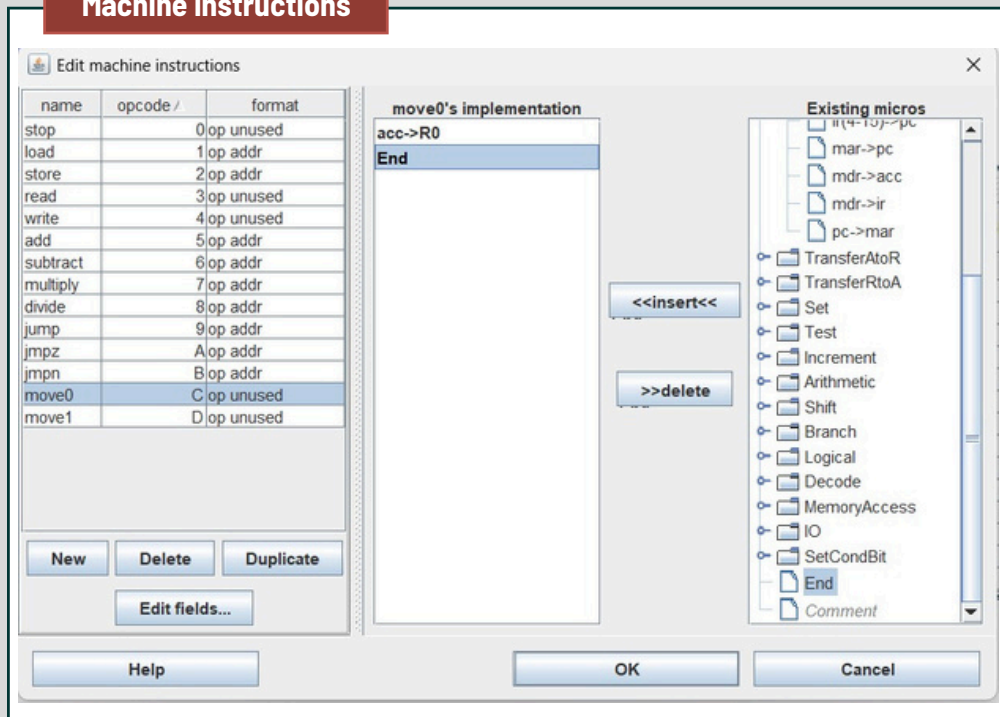
Type of Microinstruction: **TransferRtoR**

name	source	srcStartBit	dest	destStartBit	numBits
acc->R1	acc	0	R1	0	16
acc->R0	acc	0	R0	0	16
R1->acc	R1	0	acc	0	16
R0->acc	R0	0	acc	0	16
acc->mdr	acc	0	mdr	0	16
ir(4-15)->mar	ir	4	mar	0	12
ir(4-15)->pc	ir	4	pc	0	12
mar->pc	mar	0	pc	0	12
mdr->acc	mdr	0	acc	0	16
mdr->ir	mdr	0	ir	0	16
pc->mar	pc	0	mar	0	12

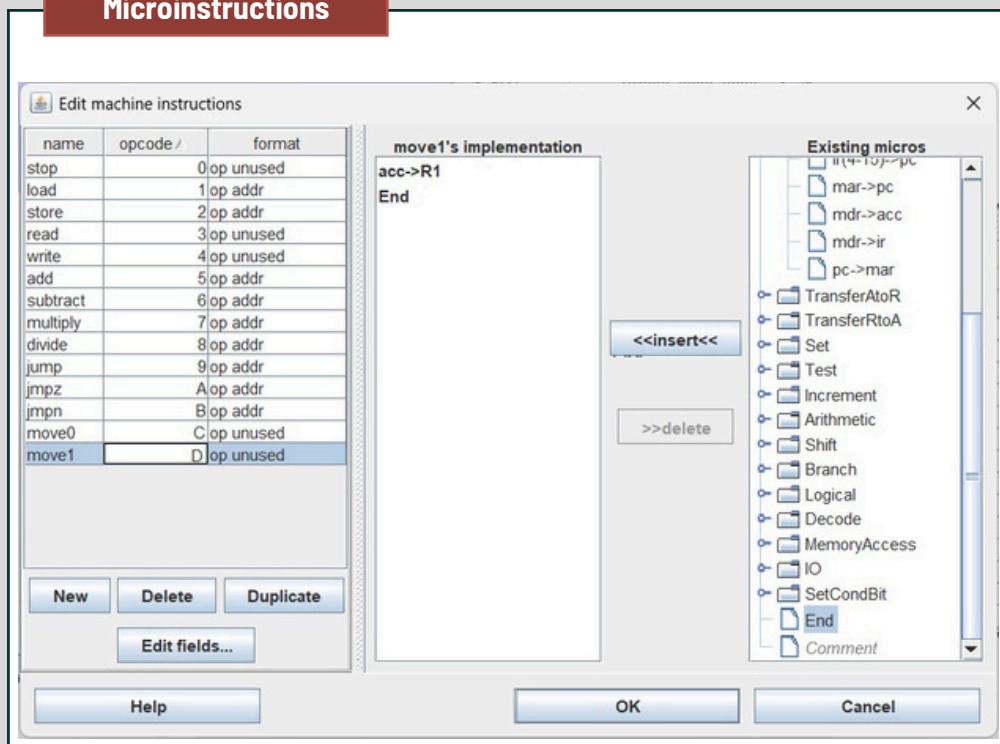
New **Delete** **Duplicate**

Help **OK** **Cancel**

Machine instructions



Microinstructions



Machine instructions

Edit machine instructions

name	opcode	format
loadAccc1	F	op unused
loadAccc0	E	op unused
move0	C	op unused
move1	D	op unused
stop	0	op unused
load	1	op addr
store	2	op addr
read	3	op unused
write	4	op unused
add	5	op addr
subtract	6	op addr
multiply	7	op addr
divide	8	op addr
jump	9	op addr
jmpz	A	op addr
jmpn	B	op addr

loadAccc1's implementation

```
R1->acc
End
```

Existing micros

- TransferRtoR
- TransferAtoR
- TransferRtoA
- Set
- Test
- Increment
- Arithmetic
- Shift
- Branch
- Logical
- Decode
- MemoryAccess
- IO
- SetCondBit
- End
- Comment

Microinstructions

Edit machine instructions

name	opcode	format
loadAccc1	F	op unused
loadAccc0	E	op unused
move0	C	op unused
move1	D	op unused
stop	0	op unused
load	1	op addr
store	2	op addr
read	3	op unused
write	4	op unused
add	5	op addr
subtract	6	op addr
multiply	7	op addr
divide	8	op addr
jump	9	op addr
jmpz	A	op addr
jmpn	B	op addr

loadAccc0's implementation

```
R0->acc
End
```

Existing micros

- TransferRtoR
- TransferAtoR
- TransferRtoA
- Set
- Test
- Increment
- Arithmetic
- Shift
- Branch
- Logical
- Decode
- MemoryAccess
- IO
- SetCondBit
- End
- Comment

Format

Edit machine fields

name	type	numBits	relativity ▾	default...	signed
operand	required	12	pcRelativePreIncr	0	☒
addr	required	12	absolute	0	☐
unused	ignored	12	absolute	0	☒
op	required	4	absolute	0	☐

Fetch Sequence

Edit the machine's fetch sequence

Implementation

```

pc->mar
Main[mar]->mdr
mdr->ir
Inc2-pc
decode-ir

```

<<insert<<

>>delete

All micros

- TransferRtoR
 - R0->acc
 - R1->acc
 - acc->R0
 - acc->R1
 - acc->mdr
 - ir(4-15)->mar
 - ir(4-15)->pc
 - mar->pc
 - mdr->acc
 - mdr->ir
 - pc->mar
- TransferAtoR
- TransferRtoA
- Set
- Test
- Increment
- Arithmetic
- Shift
- Branch

Help OK Cancel

Format

Edit machine fields

name	type	numBits	relativity ▾	default...	signed
operand	required	12	pcRelativePreIncr	0	☒
addr	required	12	absolute	0	☐
unused	ignored	12	absolute	0	☒
op	required	4	absolute	0	☐

Fetch Sequence

Edit the machine's fetch sequence

Implementation

```
pc->mar
Main[mar]->mdr
mdr->ir
Inc2-pc
decode-ir
```

<<insert<<

>>delete

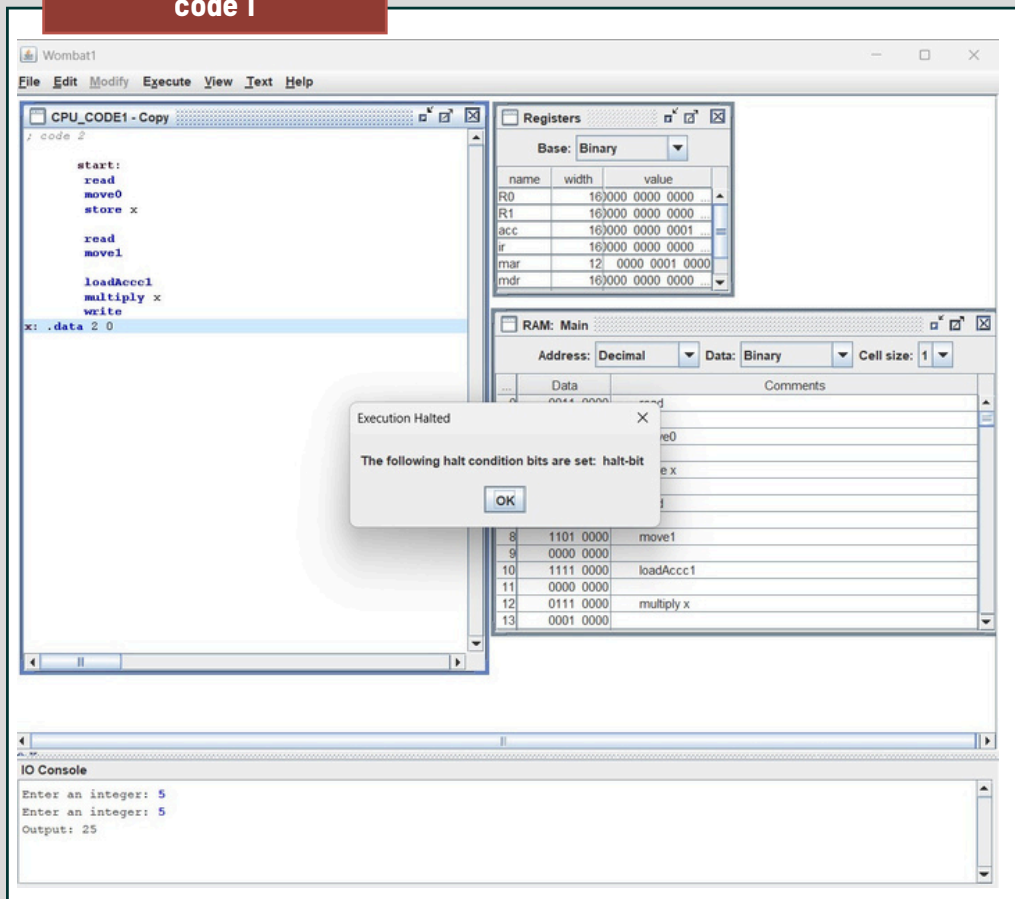
All micros

- TransferRtoR
 - R0->acc
 - R1->acc
 - acc->R0
 - acc->R1
 - acc->mdr
 - ir(4-15)->mar
 - ir(4-15)->pc
 - mar->pc
 - mdr->acc
 - mdr->ir
 - pc->mar
- TransferAtoR
- TransferRtoA
- Set
- Test
- Increment
- Arithmetic
- Shift
- Branch

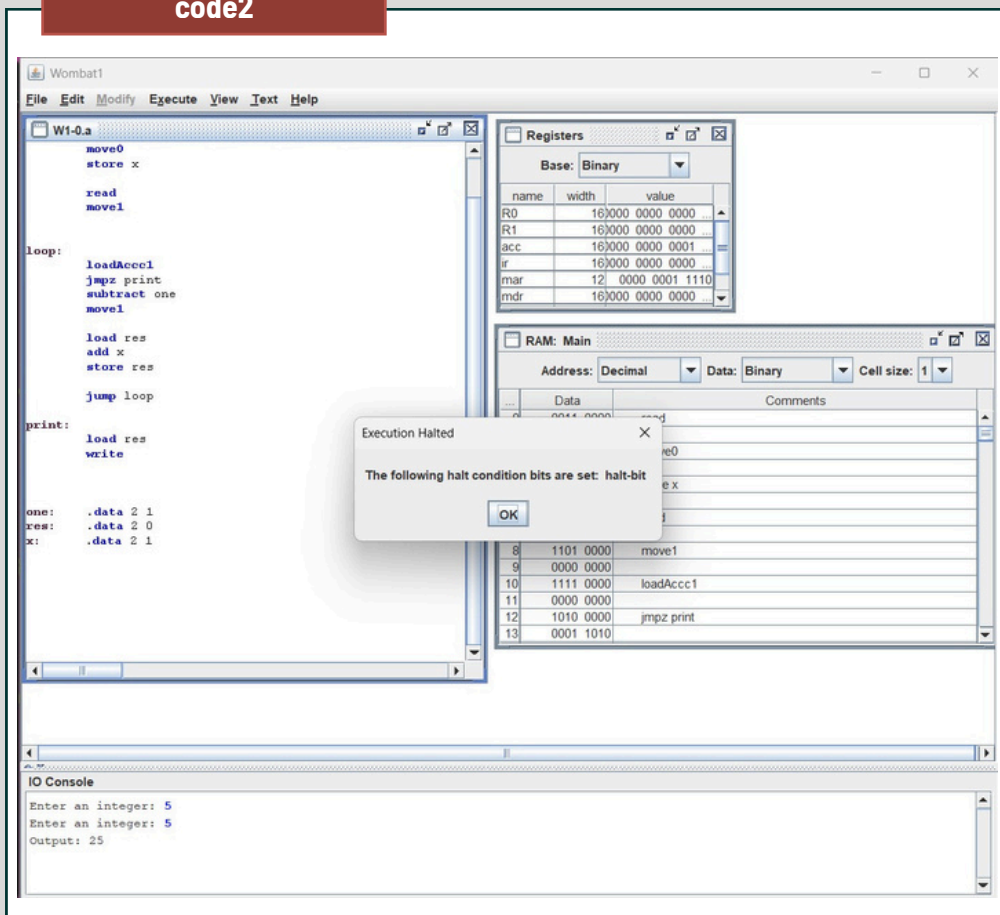
Help OK Cancel

Code Part:

code 1



code2



REPEATED ADDITION VS MULTIPLY INSTRUCTION

Multiplication by Repeated Addition (No Multiply Instruction)

This method multiplies two numbers by repeatedly adding the first number (A) as many times as the second number (B).

How the code works (Code 1):

- The first input (A) is stored in x and also saved in R0.
- The second input (B) is stored in R1, which acts as a loop counter.
- Each loop:
 - Load the counter from R1.
 - If it is zero, the program prints the result.
 - Otherwise, decrement the counter and update R1.
 - Add A to the result stored in res.
- The loop continues until the counter becomes zero.

Multiplication Using the Multiply Instruction

This method uses the CPU's built-in multiply instruction to compute the product directly.

How the code works (Code 2):

- The first number (A) is stored in memory x.
- The second number (B) is read into R1 and then loaded into ACC.
- The instruction multiply x calculates $ACC = B \times A$.
- The result in ACC is printed immediately.

COMPARISON

FEATURE	CODE 1: REPEATED ADDITION	CODE 2: MULTIPLY INSTRUCTION
Instruction Complexity	Low: Uses simple instructions such as load, store, add, subtract, and jump inside a loop.	High: Uses a dedicated multiply instruction that performs an entire arithmetic operation internally in one step.
Program Complexity	High: The programmer must manage the loop, decrement the counter (R1), update the result manually, and handle multiple control-flow jumps. The code is noticeably longer.	Low: The multiply logic is handled by the hardware. The program is short and requires only a few instructions.
Memory Consumed	Higher: Requires multiple variables (x, res, one) and more instructions, leading to larger memory usage.	Lower: Only one data variable (x) and fewer instructions, so memory consumption is minimal.

Explanation:

- **Code 1** has simpler instructions but more manual work and larger memory usage due to the loop and extra variables.
- **Code 2** has fewer instructions and smaller memory footprint, but relies on hardware support for the multiply instruction.

RISC AND CISC

ARCHITECTURE	TYPE	EXPLANATION
Repeated Addition	RISC (Reduced Instruction Set Computer)	Uses simple, basic instructions like load, store, add, sub, jump, and loops. Each instruction does a small, specific task. Multiplication is implemented by combining multiple instructions and managing the loop counter (R1) and result (res) manually (as in Code 1).
Multiply Instruction	CISC (Complex Instruction Set Computer)	Uses a single complex instruction (multiply) to perform the entire multiplication operation. Fewer instructions are needed, and the hardware handles the complexity internally (as in Code 2). Each instruction can perform a lot of work in one step.

Explanation:

RISC Emphasizes simplicity, uniform instruction length, and software-managed operations.

CISC Emphasizes powerful instructions that can perform complex tasks in hardware, reducing code size and program complexity.